

Halo 2 Guide

From a circuit to a checked proof

m@rek.onl

Abstract

This volume of *The Zcash Arboretum* constructs the Halo 2 proof system used to prove shielded transaction relations. It assumes the *Math Guide*'s fields, polynomials, linear algebra and Pasta curves, and the *Crypto Guide*'s commitments, security contracts and interactive proofs. We construct a way to convince a verifier that private data satisfies public rules without disclosing that data. The verifier's complete calculation makes explicit what it checks, what the application must supply, and which security conclusions require more than the algebraic identities.

Contents

1	From a relation to a circuit	3
1.1	The statement being proved	3
1.2	The field and the table	3
1.3	A complete small computation	4
1.4	Integers inside a field	4
1.5	The commitment interface	4
2	Columns as polynomials	5
2.1	A row is an evaluation point	5
2.2	The four-row table in coefficient form	5
2.3	Combining identities	6
2.4	Degree, cosets and quotient chunks	6
2.5	Reserved rows	7
3	The permutation argument	7
3.1	Copy constraints as a permutation	7
3.2	The running product	8
3.3	Chunking the columns	8
4	The lookup argument	9
4.1	Membership, not equal multiplicity	9
4.2	Arranging equal runs	9
4.3	Proving the two permutations	9
5	Polynomial commitments and the masked inner-product argument	10
5.1	Committing to a bounded-degree polynomial	10
5.2	The opening claim and its mask	11
5.3	One folding step	11
5.4	The final check	12

6	Reducing multiple openings	12
6.1	The list to be authenticated	12
6.2	First combine equal point sets	13
6.3	Combine the divisibility claims	13
6.4	One final polynomial	14
7	The transcript and the verifier	14
7.1	The circuit-specific keys	14
7.2	Hashing messages into challenges	15
7.3	The whole sequence	15
7.4	The acceptance algorithm	16
8	Knowledge soundness and zero knowledge	17
8.1	What the algebra has established	17
8.2	A conditional arithmetic-layer statement	17
8.3	The commitment and Fiat–Shamir boundary	18
8.4	Why polynomial evaluations need masks	18
8.5	The quotient mask and the joint transcript	19
8.6	The application-facing contract	19
8.7	Primary construction sources	20

1 From a relation to a circuit

1.1 The statement being proved

The mathematical prerequisites are the *Math Guide*, “Prime fields and modular arithmetic”, “Linear algebra and uniform masks”, “Polynomials and evaluation”, “Evaluation domains and fast transforms”, and “Pasta curves and their encodings”. We use their fields, vectors, polynomials and curve groups with the same definitions.

A proof establishes a precisely selected relation, not an informal claim that a transaction is good. Let u be a public input, w a private witness, and $\mathcal{R}(u, w)$ a deterministic predicate. The prover receives both inputs; the verifier receives u and a proof. Completeness asks that an honestly constructed proof be accepted when $\mathcal{R}(u, w) = 1$. Knowledge soundness asks for a witness-extraction guarantee against an efficient prover that succeeds sufficiently often. Zero knowledge asks that the verifier’s view be simulatable from the public input alone. These distinct contracts are defined in the *Crypto Guide*, “Security contracts and reductions” and “Knowledge, simulation, and Schnorr”.

The application chooses $\mathcal{R}$. Halo 2 represents that relation by an *arithmetic circuit*: field-valued cells connected by polynomial equations which must hold simultaneously. Its soundness cannot repair an omitted bound on a value, a wrong hash input, or a public value left unconnected to the witness. There are therefore two obligations:

$$\begin{aligned} \text{circuit satisfaction} &\implies \mathcal{R}(u, w) = 1, \\ \text{accepted proof} &\implies \text{the proof system’s knowledge guarantee.} \end{aligned}$$

The first is about the circuit. The second is about the argument system. Neither implication follows merely from the other.

We use the Zcash Halo 2 implementation, specifically `halo2_proofs` at commit

261faaccd5a30c19bb8468600841a0dac305adf5.

The pinned source is the authority for the folding and transcript conventions below. Other systems named Halo 2 can use different polynomial commitments or transcripts. The implementation sources are linked in §8.7. This construction does not verify earlier proofs inside another proof.

1.2 The field and the table

Let $\mathbb{F}$ be the field in which the circuit computes. For the relevant Pasta instantiation it is the Pallas base field, equivalently the Vesta scalar field. The commitment group used below is Vesta, not Pallas. The *Math Guide*, “Pasta curves and their encodings”, gives both fields and their encodings.

A circuit is a fixed arrangement of columns and rows. An *advice* column holds witness-dependent values. A *fixed* column holds circuit constants. An *instance* column holds public inputs, padded with zeroes to the circuit size. A *gate* is a polynomial expression in selected cells which must equal zero. A query can select the current row or a fixed rotation, such as the next row. Rows are indexed modulo the row count; a next-row query at the last row wraps to row zero.

A *verification key* is public data fixing this circuit’s columns, equations, constants and wiring. The verifier must use the intended key, not a different circuit selected by the prover. A fixed selector is a column which turns a gate on at its intended rows. For example,

$$q_{\text{mul}}(ab - c) = 0$$

imposes multiplication where the selector is one and imposes nothing where it is zero. In the implementation, selectors may be compressed into fixed columns and replaced by polynomial expressions. The verification key fixes the resulting expressions; they are not choices the prover can change after seeing a challenge.

Assignments are not constraints. Computing a value in the witness generator only tells the honest prover what to put in a cell. A malicious prover need not run that generator. Two cells assigned the

same host-language variable must still be joined by an equality constraint if their equality matters. Similarly, a witness calculated from a public input must be constrained to that input.

1.3 A complete small computation

Consider the relation

$$\mathcal{R}(t, s) = 1 \iff s^3 + s + 5 = t \text{ in } \mathbb{F}_{97}.$$

The public input is $t = 35$; one witness is $s = 3$. We use four rows and the gate

$$q_M ab + q_L a + q_R b + q_O c + q_C + \pi = 0. \quad (1)$$

The column π is public. All five q columns are fixed.

i	a	b	c	q_M	q_L	q_R	q_O	q_C	π
0	3	3	9	1	0	0	-1	0	0
1	9	3	27	1	0	0	-1	0	0
2	27	3	30	0	1	1	-1	0	0
3	30	0	0	0	1	0	0	5	-35

The negative entries denote field elements: for example $-1 = 96$. The equalities are

$$a_0 = b_0 = b_1 = b_2, \quad c_0 = a_1, \quad c_1 = a_2, \quad c_2 = a_3.$$

These make the four local equations one computation: s^2 , then s^3 , then $s^3 + s$, then comparison with $t - 5$. Without the copy equalities, the rows could describe unrelated numbers.

This four-row table isolates the arithmetic. It does not reserve blinding rows—unused rows filled with randomness to conceal witness values—or instantiate a secure curve, so it is not a production Halo 2 proof. Its purpose is to make the polynomial calculations independently reproducible. The verifier and masking rules for an actual circuit are introduced below.

1.4 Integers inside a field

The circuit sees residues, not unbounded integers. A field equality $a + b = c$ also holds when the intended integer sum differs by the field modulus. To interpret a field element as a k -bit nonnegative integer, one can constrain bits b_i by

$$b_i(b_i - 1) = 0, \quad a = \sum_{i=0}^{k-1} 2^i b_i,$$

and require 2^k not to exceed the modulus. A *lookup* checks that a value belongs to a specified table. Such table-membership checks can replace individual bit constraints when enforcing a range of integers. Bounds on intermediate sums must also rule out wraparound. The *Math Guide*, “Language, integers, and encodings” and “Prime fields and modular arithmetic”, proves the required integer-to-field statements. This obligation is essential whenever a circuit represents money or encoded byte strings.

1.5 The commitment interface

The *Crypto Guide*, “Groups, commitments, and Sinsemilla”, constructs commitments to vectors of scalars. Committing to a polynomial of degree less than n means applying that construction to its coefficient vector, padded to length n :

$$C = \sum_{i=0}^{n-1} [a_i]G_i + [r]W \quad \text{for } p(X) = \sum_{i=0}^{n-1} a_i X^i.$$

The public points G_i, W belong to a group with scalar field $\mathbb{F}$. The scalar r is the commitment blind. The lower volume's hiding and binding contracts apply to these vectors. The concrete parameter derivation and the algorithm for proving an evaluation are constructed in §5; until then, the arithmetic reductions describe the identities that such evaluations must authenticate.

The *Crypto Guide*, “Knowledge, simulation, and Schnorr”, also introduces the Fiat–Shamir transformation: deterministic hash-derived challenges replace an interactive verifier’s random challenges. We first explain the arithmetic with random challenges. A *transcript* is the ordered record of public inputs, messages and challenges. Its concrete byte construction is given in §7; its security is not assumed to follow from a single random-point calculation.

2 Columns as polynomials

2.1 A row is an evaluation point

Choose $n = 2^k$ dividing $|\mathbb{F}| - 1$ and a primitive n th root of unity ω . The evaluation domain is

$$H = \{1, \omega, \dots, \omega^{n-1}\}, \quad Z_H(X) = X^n - 1.$$

For each column with entries a_i , there is exactly one polynomial $a(X)$ of degree less than n satisfying $a(\omega^i) = a_i$. The *Math Guide*, “Polynomials and evaluation”, constructs it as $\sum_i a_i L_i(X)$, where

$$L_i(X) = \frac{\omega^i X^n - 1}{n X - \omega^i}.$$

The quotient is a polynomial; the displayed rational expression is evaluated by cancellation at a domain point. A rotation by r rows is the polynomial $a(\omega^r X)$. It has the same degree as a .

Substituting the column polynomials into a gate produces a polynomial $g(X)$. Its row constraints hold exactly when g vanishes on H . Since the elements of H are distinct,

$$\forall h \in H, g(h) = 0 \iff \exists q(X) : g(X) = Z_H(X)q(X). \quad (2)$$

This is polynomial divisibility, not pointwise division on H , where the denominator would be zero.

2.2 The four-row table in coefficient form

In the toy example, choose $\omega = 22$. Its successive powers modulo 97 are 1, 22, 96, 75. Interpolation gives

$$\begin{aligned} a(X) &= 90 + 61X + 22X^2 + 24X^3, \\ b(X) &= 75 + 32X + 25X^2 + 65X^3, \\ c(X) &= 65 + 16X + 3X^2 + 22X^3. \end{aligned}$$

Interpolate the fixed and public columns in exactly the same way and substitute them into (1). The resulting gate polynomial is

$$g(X) = (X^4 - 1)(61 + 70X + 43X^2 + 47X^3 + 81X^4 + 46X^5).$$

Every coefficient and the following evaluations are checked by `research/check_foundations.py`. At the off-domain point $x = 20$,

$$Z_H(20) = 46, \quad q(20) = 67, \quad g(20) = 75, \quad 46 \cdot 67 = 75 \pmod{97}.$$

Now change only c_0 from 9 to 10. Division of the new gate polynomial by $X^4 - 1$ leaves remainder

$$r(X) = 24(1 + X + X^2 + X^3).$$

Its roots in $\mathbb{F}_{97}$ are 22, 75, 96: exactly the three unchanged rows. At $x = 20$, the altered numerator is 20, while its Euclidean quotient times $Z_H(20)$ is 64. The test rejects. The changed table also breaks a copy equality, but this calculation concerns the gate check alone.

For this particular remainder, a uniform field point is a root with probability $3/97$. That is not a security estimate for Halo 2. The field is deliberately tiny, the commitment argument has not yet been applied, and the production verifier has exceptional cases such as $x \in H$ which must be accounted for separately.

2.3 Combining identities

Suppose the circuit and its auxiliary arguments require $g_0, \dots, g_{M-1}$ to vanish on H . After their defining commitments are fixed, sample a challenge y and combine them in Horner order:

$$N_y(X) = y^{M-1}g_0(X) + y^{M-2}g_1(X) + \dots + g_{M-1}(X).$$

The order is the verification key's gate order, followed by permutation and lookup constraints, repeated for each circuit instance in a joint proof.

If one row violates any of these identities, at that row the combined expression is a nonzero polynomial in y of degree at most $M - 1$. For fixed constraints and an independent uniform challenge, accidental cancellation has probability at most $(M - 1)/|\mathbb{F}|$, by the root bound in the *Math Guide*, “Polynomials and evaluation”. This is a local interactive argument. An actual Fiat–Shamir security bound must also account for how the adversary obtains and retries challenges.

After receiving y , the prover commits to a quotient h intended to satisfy

$$N_y(X) = (X^n - 1)h(X).$$

The verifier subsequently samples x and checks the equality at x . If the difference polynomial is nonzero and has degree at most D , an independent uniform x hides that difference with probability at most $D/|\mathbb{F}|$. The timing matters: choosing h after learning x would allow a prover to fit one point without satisfying the identity.

2.4 Degree, cosets and quotient chunks

Let d be the configured constraint-degree bound. It is the maximum of the permutation and lookup requirements, all gate degrees, and any explicitly configured minimum degree. In counting expression degrees, every queried column counts as degree one; selectors and boundary polynomials count too. Thus d need not be attained by an expression. Substituting degree-less-than- n column polynomials gives

$$\deg N_y \leq d(n - 1).$$

For a satisfying witness, division by the degree- n polynomial Z_H therefore gives $\deg h < (d - 1)n$. Write

$$h(X) = \sum_{j=0}^{d-2} X^{jn} h_j(X), \quad \deg h_j < n. \quad (3)$$

Each chunk now fits the same commitment parameters as a column. There are always $d - 1$ chunk commitments, including any zero high chunks; proof shape does not reveal the actual degree.

The prover evaluates the constraints on an extended coset of size

$$N = 2^{\lceil \log_2((d-1)n) \rceil}.$$

The implementation uses $\zeta \langle \omega_{\text{ext}} \rangle$, where ζ has order three and ω_{ext} has order N . The coset is disjoint from H , since a nontrivial element of order three cannot belong to a group of power-of-two order. Pointwise division by $X^n - 1$ is therefore defined there. An inverse transform recovers the quotient coefficients. The *Math Guide*, “Evaluation domains and fast transforms”, supplies these transforms.

The extended size accommodates the *quotient*, not necessarily the numerator. For $d = 9$, it is $8n$, although the numerator can have degree close to $9n$. Evaluating the constraint expression directly on the coset and dividing is sufficient for an honest witness. For an invalid witness, the interpolated ratios need not be its Euclidean quotient; acceptance still requires the independently challenged identity above.

Once x is known, the verifier forms the commitment to

$$h_x(X) = \sum_{j=0}^{d-2} (x^n)^j h_j(X).$$

The coefficients $(x^n)^j$ are now scalars. This new polynomial has degree less than n , and $h_x(x) = h(x)$. Its expected value is $N_y(x)/(x^n - 1)$, calculated from the other claimed evaluations. The proof does not separately transmit each $h_j(x)$.

2.5 Reserved rows

Let b be the number of blinding factors, and put $\ell = n - b - 1$. Rows $0, \dots, \ell - 1$ are usable. Row ℓ holds the final value of an auxiliary running product. Rows $\ell + 1, \dots, n - 1$ are product blinding rows. Define

$$L_{\text{blind}}(X) = \sum_{i=\ell+1}^{n-1} L_i(X), \quad A_{\text{act}}(X) = 1 - L_{\ell}(X) - L_{\text{blind}}(X).$$

On H , the active-row polynomial is one exactly at usable rows.

Advice columns are randomised at *all* $b + 1$ unusable rows, including row ℓ . Product polynomials retain their constrained value at row ℓ and randomise only the last b rows. Gates must be disabled where they would impose unintended constraints on these random cells. A rotation must likewise not make an active gate depend on a random reserved cell unless that dependence is intended.

3 The permutation argument

3.1 Copy constraints as a permutation

Copy equalities partition selected cells into classes whose values must agree. Their endpoints must lie in the usable rows $0, \dots, \ell - 1$; key generation rejects wiring to any other row. Number the equality-enabled columns $j = 0, \dots, m - 1$. Label the cell in row i , column j , by

$$\text{id}_{j,i} = \delta^j \omega^i.$$

Choose δ so that these column cosets are disjoint. Halo 2 uses the field's designated DELTA; the circuit must fit the supply of disjoint cosets. Distinct labels identify distinct cells even when their witness values happen to coincide.

For each equality class, choose one cycle through its cells. Cells outside a nontrivial class are fixed points. Together these cycles define a permutation σ . The fixed polynomial s_j contains the label of the successor:

$$s_j(\omega^i) = \text{id}_{\sigma(j),i}.$$

Both the wiring and the commitments to these polynomials belong to the verification key.

Lemma 3.1 (Why a labelled product checks equality). *Let v_c be the value at cell c , and let λ_c be its distinct label. The formal identity*

$$\prod_c (v_c + \beta \lambda_c + \gamma) = \prod_c (v_c + \beta \lambda_{\sigma(c)} + \gamma)$$

holds as a polynomial in β, γ if and only if values are constant on every cycle of σ .

Proof. Fix a cell c and substitute $\gamma = -v_c - \beta\lambda_c$. The left product is zero. The right becomes a product of polynomials in β , so one of its factors must be the zero polynomial: the product of nonzero polynomials over a field is nonzero. Its coefficient of β requires $\lambda_{\sigma(d)} = \lambda_c$ for that factor's cell d , and its constant term requires $v_d = v_c$. Distinct labels therefore give $d = \sigma^{-1}(c)$. Doing this for every c proves constancy on each cycle. Conversely, these equalities merely reorder the factors. The polynomial facts used here are proved in the *Math Guide*, “Polynomials and evaluation”. $\square$

For actual random field values β, γ , a nonzero difference can vanish accidentally. The polynomial root bound controls that event; the product identity is not an unconditional test at one challenge pair. The advice commitments precede these challenges.

3.2 The running product

For one set of equality columns, an honest prover starts with $Z(1) = 1$ and, for usable rows, computes

$$Z(\omega^{i+1}) = Z(\omega^i) \frac{\prod_j (a_j(\omega^i) + \beta \delta^j \omega^i + \gamma)}{\prod_j (a_j(\omega^i) + \beta s_j(\omega^i) + \gamma)}.$$

This formula describes the ordinary case in which all denominator factors are nonzero. It telescopes: the final product equals one precisely when the two labelled products agree.

The verifier does not receive all these intermediate values. It checks a polynomial recurrence, multiplied by A_{act} :

$$\begin{aligned} A_{\text{act}}(X) \left[Z(\omega X) \prod_j (a_j(X) + \beta s_j(X) + \gamma) \right. \\ \left. - Z(X) \prod_j (a_j(X) + \beta \delta^j X + \gamma) \right] = 0. \end{aligned} \quad (4)$$

Boundary constraints fix the initial and final values. These expressions join the combined numerator N_γ ; checking them uses the same quotient machinery as checking an arithmetic gate.

3.3 Chunking the columns

A product over m columns would make (4) have expression degree $m + 2$: one for the active selector, one for Z , and m factors. For a circuit degree d , the implementation therefore puts at most $d - 2$ columns in a chunk. Let J_t be the columns of chunk t , and let Z_t be its product polynomial.

The first chunk starts at one. Each subsequent chunk starts at the previous chunk's final usable product. The exact boundary constraints are

$$L_0(X)(1 - Z_0(X)) = 0, \quad (5)$$

$$L_0(X)(Z_t(X) - Z_{t-1}(\omega^\ell X)) = 0 \quad (t > 0), \quad (6)$$

$$L_\ell(X)(Z_{T-1}(X)^2 - Z_{T-1}(X)) = 0. \quad (7)$$

Each chunk also has recurrence (4) with $j \in J_t$. Here T is the number of chunks. At row zero, $\omega^\ell X$ addresses row ℓ , which explains the rotated query in (6).

The final constraint is *Boolean*: it permits zero or one. It is not written as $Z_{T-1} = 1$. Off the exceptional set where a numerator or denominator factor vanishes, every running product is nonzero, so this Boolean constraint forces one. A soundness argument must include the exceptional event rather than silently replace the implemented constraint by a stronger one.

The verifier needs each Z_t at x and ωx , and all but the last at $\omega^\ell x$. The latter is also the rotation $-(b + 1)$, because $\omega^n = 1$. It needs every fixed s_j at x . At degree nine, fifteen equality-enabled columns require three seven-column-capacity chunks, not two. Columns can be advice, instance or fixed; counting advice columns alone is insufficient.

4 The lookup argument

4.1 Membership, not equal multiplicity

A lookup requires a row's tuple to occur among the rows of a table. For example, a table containing $0, \dots, 255$ constrains a value to one byte without eight separate bit equations. A table row can be used repeatedly. A *multiset* records values together with their multiplicities, meaning how many times each occurs. The input multiset therefore need not have the same multiplicities as the table.

Let the input tuple be $(a_0, \dots, a_{r-1})$ and the table tuple $(s_0, \dots, s_{r-1})$. After committing to the advice columns, sample θ and compress each tuple in Horner order:

$$A = \sum_{j=0}^{r-1} \theta^{r-1-j} a_j, \quad S = \sum_{j=0}^{r-1} \theta^{r-1-j} s_j.$$

A pair of distinct, fixed tuples collides under this compression with probability at most $(r-1)/|\mathbb{F}|$. To protect all relevant pairs, the analysis must count them and combine their bounds. The single-pair bound is not the error of the whole lookup argument.

Lookup expressions exist at every usable row. Disabling a conditional lookup must map the inactive input to a tuple actually present in the table, often an all-zero tuple. A selector which merely sets a value to zero is not enough if zero is absent from the table.

4.2 Arranging equal runs

The prover permutes the compressed input values into A' with equal values adjacent. It permutes the table values into S' so that the first row of each equal run in A' matches the table value on that row. Unused table entries fill the remaining positions.

For example, with input and table values

$$A = (3, 1, 3, 3), \quad S = (1, 2, 3, 4),$$

a valid arrangement is

$$A' = (1, 3, 3, 3), \quad S' = (1, 3, 2, 4).$$

The value three appears in three input rows but only one table row. At the first three, the table matches. At the next two, equality to the preceding input suffices.

The conditions are

$$A'_0 = S'_0, \quad (A'_i - S'_i)(A'_i - A'_{i-1}) = 0 \quad (0 \leq i < \ell).$$

At row zero the separate first-row equation supplies the match, regardless of the wrapped previous-row value. At a later row, either the current table entry matches or the preceding input is the same. Induction from row zero shows that every input belongs to the table, provided A' and S' really are permutations of their originals. Sorting is an honest-prover construction; the verifier does not need to prove a numerical ordering.

4.3 Proving the two permutations

The prover commits to A' and S' after θ but before the independent challenges β, γ . It forms a product Z whose ordinary-case recurrence is

$$Z(\omega^{i+1}) = Z(\omega^i) \frac{(A(\omega^i) + \beta)(S(\omega^i) + \gamma)}{(A'(\omega^i) + \beta)(S'(\omega^i) + \gamma)}.$$

The two challenges keep the input and table multiset tests separate. As a formal polynomial identity, equality of the products in β, γ implies equality of each respective multiset: compare the coefficient

of the highest power of γ to isolate the input product, and the highest power of β to isolate the table product. The one-variable fact follows by matching the monic linear factors.

The exact constraint list is

$$L_0(1 - Z) = 0, \quad (8)$$

$$L_\ell(Z^2 - Z) = 0, \quad (9)$$

$$A_{\text{act}}[Z(\omega X)(A' + \beta)(S' + \gamma) - Z(X)(A + \beta)(S + \gamma)] = 0, \quad (10)$$

$$L_0(A' - S') = 0, \quad (11)$$

$$A_{\text{act}}(A' - S')(A' - A'(\omega^{-1}X)) = 0. \quad (12)$$

Unmarked polynomials in these equations are evaluated at X . As in the permutation argument, the Boolean endpoint forces one only off the zero-factor exceptional set.

The prover randomises A' and S' in the last $b + 1$ rows. It randomises Z in the last b rows while retaining its endpoint at row ℓ . The verifier needs five evaluations per lookup:

$$Z(x), \quad Z(\omega x), \quad A'(x), \quad A'(\omega^{-1}x), \quad S'(x).$$

Together with evaluations of the original input and table expressions, these suffice to compute all five constraints at x . Authentication of those evaluations is the remaining task.

5 Polynomial commitments and the masked inner-product argument

5.1 Committing to a bounded-degree polynomial

Let $\mathbb{G}$ be a prime-order group whose scalar field is $\mathbb{F}$. Its identity is $\mathcal{O}$. Choose public generators $G_0, \dots, G_{n-1}, W, U \in \mathbb{G}$ without known efficiently computable discrete logarithm relations. For $p(X) = \sum_{i=0}^{n-1} a_i X^i$ and blind $r \in \mathbb{F}$, define

$$\text{Commit}(p; r) = \sum_{i=0}^{n-1} [a_i]G_i + [r]W.$$

This is the vector commitment from the *Crypto Guide*, “Groups, commitments, and Sinsemilla”. We use $\langle a, G \rangle = \sum_i [a_i]G_i$ for a scalar–point inner product, and $\langle a, b \rangle = \sum_i a_i b_i$ for a scalar–scalar inner product.

A uniform independent r makes the commitment perfectly hiding: multiplication by nonidentity W is a bijection from $\mathbb{F}$ to $\mathbb{G}$. Binding is computational, not information-theoretic. The map from $n + 1$ scalars to one group element necessarily has many collisions; the assumption is that a bounded adversary cannot find the relevant distinct openings. Binding alone does not supply an extractor for a prover that sends group elements without their representations.

Halo 2 derives its parameters deterministically. With the Pasta hash-to-curve construction from the *Crypto Guide*, “Hash functions and key derivation”, using domain `Hal02-Parameters`, it hashes

point	message bytes
G_i	<code>0x00</code> <code>LE32_{bytes}(i)</code>
W	<code>0x01</code>
U	<code>0x02</code>

where `LE32bytes` encodes an integer in four little-endian bytes. This needs no secret setup ceremony. It does not remove the assumptions about the resulting points or their derivation.

For a column already stored as evaluations $p(\omega^i)$, precompute Lagrange-basis generators

$$G_i^{\text{Lag}} = \frac{1}{n} \sum_{j=0}^{n-1} [\omega^{-ij}]G_j.$$

Here the factor $1/n$ also acts by scalar multiplication. Inverse Fourier interpolation gives

$$\text{Commit}(p; r) = \sum_i [p(\omega^i)] G_i^{\text{Lag}} + [r]W.$$

Both bases therefore commit to the same polynomial. Fixed and instance commitments use the implementation's known default blind, which is *one*; it is not a fresh secret mask and not zero.

5.2 The opening claim and its mask

An opening claim consists of a commitment P , a point $x \in \mathbb{F}$ and a value $v \in \mathbb{F}$. The honest prover knows (p, r) with

$$P = \text{Commit}(p; r), \quad p(x) = v, \quad \deg p < n.$$

The desired verification statement combines commitment consistency with evaluation correctness. Sending all coefficients would prove the identity but disclose the polynomial and cost n scalars.

The implemented opening starts by sampling a uniform degree-less-than- n polynomial s_0 and setting

$$s(X) = s_0(X) - s_0(x).$$

Thus $s(x) = 0$. This is a root at the opening point x , not necessarily at zero. The map $s_0 \mapsto s$ is onto the kernel of evaluation at x , with exactly $|\mathbb{F}|$ preimages per output, so s is uniform in that $(n - 1)$ -dimensional subspace. The linear-map facts are proved in the *Math Guide*, “Linear algebra and uniform masks”.

The prover samples an independent blind r_s and sends $S = \text{Commit}(s; r_s)$. The transcript then produces challenges ξ and z , in that order. Define

$$\hat{p} = p - v + \xi s, \quad \hat{r} = r + \xi r_s, \quad \hat{P} = P - [v]G_0 + [\xi]S.$$

The honest claim now says $\hat{P} = \text{Commit}(\hat{p}; \hat{r})$ and $\hat{p}(x) = 0$. For a fixed nonzero ξ independent of s , the polynomial $p - v + \xi s$ is uniform in the zero-evaluation subspace, regardless of which p with evaluation v was used. This is a local masking fact, not yet a simulator for an adaptively hashed transcript.

5.3 One folding step

Let a be the coefficients of $\hat{p}$, and let

$$b = (1, x, \dots, x^{n-1}), \quad f = \hat{r}.$$

Initially the verifier's group expression is $M = \hat{P}$. The invariant to preserve is

$$M = \langle a, G \rangle + [z\langle a, b \rangle]U + [f]W. \tag{13}$$

The inner product is initially zero, so this invariant holds for an honest opening.

Split each vector into equal left and right halves, denoted L, R . Sample fresh independent blinds λ, ρ , and send

$$\begin{aligned} L_j &= \langle a_R, G_L \rangle + [z\langle a_R, b_L \rangle]U + [\lambda]W, \\ R_j &= \langle a_L, G_R \rangle + [z\langle a_L, b_R \rangle]U + [\rho]W. \end{aligned}$$

The transcript supplies challenge u_j . For $u_j \neq 0$, fold by

$$\begin{aligned} a' &= a_L + u_j^{-1}a_R, & b' &= b_L + u_j b_R, \\ G' &= G_L + [u_j]G_R, & f' &= f + u_j^{-1}\lambda + u_j\rho, \\ M' &= M + [u_j^{-1}]L_j + [u_j]R_j. \end{aligned}$$

Vector operations are componentwise.

Lemma 5.1 (The fold preserves the invariant). *If (13) holds, then the same identity holds for the primed vectors and scalars.*

Proof. Expand the folded scalar–point inner product:

$$\begin{aligned}\langle a', G' \rangle &= \langle a_L, G_L \rangle + \langle a_R, G_R \rangle \\ &\quad + [u_j] \langle a_L, G_R \rangle + [u_j^{-1}] \langle a_R, G_L \rangle.\end{aligned}$$

The scalar inner product has the identical expansion with b in place of G . The two cross terms are exactly those added by $[u_j^{-1}]L_j + [u_j]R_j$, including their blind contributions. The remaining terms are the old invariant. $\square$

This asymmetric convention uses u_j and u_j^{-1} , not their squares. Changing to a symmetric convention without changing every equation would produce a different protocol.

5.4 The final check

After $k = \log_2 n$ rounds, each vector has length one. The prover sends $c = a_{\text{final}}$ and $f = f_{\text{final}}$. The verifier checks

$$\hat{P} + \sum_{j=0}^{k-1} ([u_j^{-1}]L_j + [u_j]R_j) = [c]G_{\text{final}} + [czb_{\text{final}}]U + [f]W. \quad (14)$$

It calculates the folded public quantities without receiving them. Define the degree- $n-1$ polynomial

$$B(X) = \prod_{j=0}^{k-1} (1 + u_j X^{n/2^{j+1}}) = \sum_{i=0}^{n-1} B_i X^i.$$

Then

$$b_{\text{final}} = B(x), \quad G_{\text{final}} = \sum_{i=0}^{n-1} [B_i]G_i.$$

The first split separates coefficient indices by $n/2$, the next by $n/4$, and so on, which gives this factor order. For $n = 4$, for example, the coefficient weights are $(1, u_1, u_0, u_0 u_1)$.

The opening proof contains S , two group elements per round, and the two final scalars. Its communication is logarithmic in n , but the ordinary verifier still computes a linear-size multiscalar multiplication to obtain the final generator contribution. A multiscalar multiplication is a sum of specified scalar multiples of points; batching such sums does not itself verify a proof inside another proof.

The implementation returns a deferred verification object called a Guard. Its challenge-dependent generator contribution must still be supplied and the resulting multiscalar multiplication evaluated to the identity. The single-proof verification strategy does both. Constructing the deferred object alone is not proof acceptance.

The algebra above excludes zero challenges where an inverse is needed; a zero z or ξ also removes a needed security contribution. These are exceptional events in the analysis. The pinned transcript does not implement an implicit retry-until-nonzero convention. An implementation failure or rejection on an exceptional transcript must not be described as resampling.

6 Reducing multiple openings

6.1 The list to be authenticated

The gate, permutation and lookup checks query many polynomials at several rotations of x . The verifier has an ordered list

$$(P_i, t_i, v_i),$$

where each P_i names a particular committed polynomial, t_i is an evaluation point, and v_i is the claimed value. Repeated references to one polynomial use the same identity. Each polynomial–point pair occurs only once in this input list. The verification key and the argument construction determine that list, not an unauthenticated prover-supplied description.

Different polynomials can have different point sets. For example, an advice column might be queried only at x , while a permutation product is queried at $x, \omega x, \omega^\ell x$. Opening each query separately would repeat the inner-product argument. Halo 2 reduces all the queries to one opening using polynomial interpolation and linear commitments.

6.2 First combine equal point sets

Collect the distinct points at which each polynomial is queried. Group polynomials with identical point sets into groups $j = 0, \dots, m-1$. The implementation preserves first occurrence of each polynomial in the query list; groups are ordered by the first polynomial belonging to each group. Point order follows first occurrence in the original list. These rules make both sides derive the same order.

After the original evaluations have entered the transcript, derive x_1 and x_2 . If group j contains polynomials $p_{j,0}, \dots, p_{j,k_j-1}$ in that order, define

$$Q_j(X) = \sum_{i=0}^{k_j-1} x_1^{k_j-1-i} p_{j,i}(X).$$

Use the same Horner combination for their commitments, blinds and claimed values at every point of the shared set T_j . For instance, three polynomials combine as $x_1^2 p_0 + x_1 p_1 + p_2$, not $p_0 + x_1 p_1 + x_1^2 p_2$.

Let r_j be the unique interpolation polynomial of degree less than $|T_j|$ agreeing with these combined claimed values, and let

$$V_j(X) = \prod_{t \in T_j} (X - t).$$

The entire grouped claim is equivalent to

$$V_j(X) \mid Q_j(X) - r_j(X).$$

The *Math Guide*, “Polynomials and evaluation”, constructs both interpolation and division, so this is an explicit polynomial test, not an assumed new commitment primitive.

6.3 Combine the divisibility claims

For honest claims, set

$$f_j(X) = \frac{Q_j(X) - r_j(X)}{V_j(X)}, \quad Q'(X) = \sum_{j=0}^{m-1} x_2^{m-1-j} f_j(X).$$

Every quotient has degree less than n . The prover commits to Q' with a fresh independent blind and sends this one commitment.

An implementation can compute f_j by repeated synthetic division of Q_j by the factors $X - t$, discarding remainders. The resulting quotient is also the quotient of $Q_j - r_j$: the unique remainder upon division by V_j is precisely the interpolation polynomial of Q_j ’s actual values. If a claimed value is wrong, the prover’s actual remainder and the verifier’s claimed interpolation differ. The next check is where that discrepancy must be caught.

After the commitment to Q' , derive x_3 . The prover sends

$$u_j = Q_j(x_3) \quad (0 \leq j < m).$$

Provided $x_3 \notin \bigcup_j T_j$, the verifier computes the expected value

$$v' = \sum_{j=0}^{m-1} x_2^{m-1-j} \frac{u_j - r_j(x_3)}{V_j(x_3)}. \quad (15)$$

The claim is that $Q'(x_3) = v'$. The challenge x_3 is sampled and the claims u_j are transmitted only after the commitment to Q' . Some values can already be predictable, for example for a constant Q_j ; the required order concerns the challenge and messages.

For a singleton set $T_j = \{t\}$, interpolation is just the constant claimed value $r_j = v_j$, and the summand is $(u_j - v_j)/(x_3 - t)$. For two points t_0, t_1 , it is obtained using

$$r_j(X) = v_{j,0} \frac{X - t_1}{t_0 - t_1} + v_{j,1} \frac{X - t_0}{t_1 - t_0}, \quad V_j(X) = (X - t_0)(X - t_1).$$

Thus both the single-point and rotated-quy cases use the same construction.

6.4 One final polynomial

Only after the u_j values are in the transcript, derive x_4 . Starting with $P_{\text{final}} = Q'$, perform the Horner update $P_{\text{final}} \leftarrow x_4 P_{\text{final}} + Q_j$ for groups in order. Equivalently,

$$P_{\text{final}}(X) = x_4^m Q'(X) + \sum_{j=0}^{m-1} x_4^{m-1-j} Q_j(X).$$

Its commitment is the same linear combination of known commitments; no extra final commitment is sent. The verifier calculates the claimed evaluation

$$v_{\text{final}} = x_4^m v' + \sum_{j=0}^{m-1} x_4^{m-1-j} u_j.$$

The prover now supplies the masked inner-product opening of P_{final} at x_3 to v_{final} , exactly as in §5. Linear combination of commitment blinds gives the prover's final opening blind.

Each challenge has a specific job. The challenge x_1 prevents fixed errors among polynomials in a group from cancelling deliberately; x_2 combines the quotient identities; x_3 tests their agreement at a fresh point; x_4 combines the last evaluation claims. These explanations rely on earlier objects being fixed and on the eventual opening argument having the necessary soundness guarantee. They do not turn computational binding by itself into knowledge extraction.

Point coincidences, including $x_3 \in T_j$, belong to the exceptional set. Their actual implementation handling is not a licence to divide by zero in the proof of correctness. Nor may an analysis assume that every differently named rotation is distinct when $x = 0$.

7 The transcript and the verifier

7.1 The circuit-specific keys

The public parameters depend on the field, group and domain size. A verification key additionally fixes the circuit: column counts, query rotations, gate and lookup expressions, equality wiring, fixed commitments and degree. A proving key contains that verification key and precomputed polynomial forms needed by the prover. Key generation does not require the private transaction witness. The resulting circuit shape must not change with the witness.

The verifier must obtain the intended verification key through its application's authenticated rules. Accepting a prover-selected key would allow a proof for a different, weaker relation. Fixing the key also fixes the meaning and position of every public input.

In the pinned implementation the verification-key transcript representative is formed by taking the Rust debug-format string of the key’s *pinned* contents, prefixing its UTF-8 byte length as an eight-byte little-endian integer, hashing with 64-byte BLAKE2b under personalisation `Hal02-Verify-Key`, and reducing the 64-byte little-endian result modulo the scalar-field order. The pinned contents include both field moduli, domain parameters, constraint system, fixed commitments and permutation key. This is a version-specific circuit identifier, not a claim that arbitrary Rust debug formatting is a portable consensus serialisation.

The verifier constructs each public instance polynomial by zero-padding the supplied column to n entries, with at most ℓ supplied entries. It computes its commitment with known blind one. The verification-key representative and these instance commitments enter the transcript as common inputs, without being transmitted as proof bytes.

7.2 Hashing messages into challenges

The *Crypto Guide*, “Hash functions and key derivation”, constructs BLAKE2b. Halo 2 initialises its transcript with 64-byte BLAKE2b output and personalisation `Hal02-Transcript`, padded to the hash’s personalisation-field length. The running hash state absorbs the following typed messages:

message	bytes absorbed
scalar a	$0x02 \parallel a^*$
point P	$0x01 \parallel x(P)^* \parallel y(P)^*$
challenge request	$0x00$

Scalar and coordinate encodings here are canonical 32-byte little-endian field encodings. On a challenge request, the state first absorbs the zero byte, then a clone of the state is finalised. Reduce the resulting 64-byte little-endian integer modulo $|\mathbb{F}|$ to obtain the challenge. The original state retains the newly absorbed zero byte; the hash output itself is not appended to it.

Proof bytes and hashed bytes are not identical for points. The proof contains the canonical 32-byte compressed point, whereas the transcript absorbs its two affine coordinates. Decoding must succeed before the coordinates can be hashed. The transcript rejects the identity point, which has no affine coordinates, even though the curve encoding format can represent that identity. A scalar in the proof must also decode canonically; reducing an out-of-range proof encoding would be a different parser.

The point and scalar prefix bytes separate their meanings in the hash input. Circuit and instance binding comes from the common-input prefix. Challenge ordering comes from the prescribed message sequence, not from serialising the names θ, β and so on.

Reduction of a uniformly random 512-bit integer is negligibly biased rather than exactly uniform unless the field order divides 2^{512} . The local algebraic probability calculations use ideal uniform challenges. A concrete security estimate includes this statistical difference as well as the hash model and the number of adversarial hash queries.

7.3 The whole sequence

Before combining constraints with y , the prover additionally samples a uniform degree-less-than- n polynomial R and an independent commitment blind, and sends $C_R = \text{Commit}(R; r_R)$. It later sends $R(x)$. Both R and the collapsed quotient h_x are included among the polynomials opened at x . This masks information in the later multi-opening reduction. The polynomial R is not added to the gate numerator or to the quotient identity.

The transcript order is:

Common inputs or prover messages	Next challenges
Verification-key representative; instance commitments	None
Advice commitments	θ
Permuted lookup commitments (A', S')	β, γ
Permutation products; lookup products; C_R	y
Quotient chunk commitments	x
Column and auxiliary evaluations	x_1, x_2
Combined quotient commitment $C_{Q'}$	x_3
Group evaluations $u_0, \dots, u_{m-1}$	x_4
Opening mask commitment S	ξ, z
Folding pair L_j, R_j , once per round	u_j
Final scalars c, f	Final group check

For several circuit instances proved together, each stage includes all instances before proceeding to the next challenge. Fixed columns, fixed permutation commitments, the random polynomial and the aggregate quotient are shared where the construction specifies; advice and auxiliary products remain per-instance. This shares one argument over several witnesses, not a proof that verifies earlier proofs.

The scalar evaluation messages after x are ordered as follows:

1. Instance-column queries, in configured order for each instance.
2. Advice-column queries, likewise.
3. Shared fixed-column queries.
4. The scalar $R(x)$.
5. The fixed permutation-polynomial values $s_j(x)$.
6. For each instance and each permutation chunk, the values at $x, \omega x$, then at $\omega^\ell x$ unless it is the last chunk.
7. For each instance and lookup, the five values $Z(x), Z(\omega x), A'(x), A'(\omega^{-1}x), S'(x)$.

There is no separate message for the expected quotient value $N_y(x)/(x^n - 1)$.

The multi-opening query list has its own deterministic order. For each instance it lists instance queries, advice queries, permutation queries, then lookup queries; it then appends the shared fixed queries, fixed permutation queries, and finally h_x and R at x . Permutation queries first list $(x, \omega x)$ for each chunk, then the $\omega^\ell x$ queries for the preceding chunks in reverse order. Each lookup lists Z, A', S' at x , then A' at $\omega^{-1}x$, then Z at ωx . This order determines the point-set grouping and Horner weights in §6; it need not equal the scalar transmission order.

7.4 The acceptance algorithm

Given the authenticated verification key, parameters, public instance columns and a proof byte slice, the verifier performs the following logical steps.

1. Check instance column counts and lengths. Construct the common inputs and initialise the transcript.
2. Read canonical proof elements in the fixed sequence, rejecting malformed encodings and read failures. Derive each challenge only after the preceding messages have been absorbed.
3. Evaluate every gate expression at x using the claimed column evaluations. Evaluate every permutation and lookup constraint using its claimed auxiliary evaluations and the public $L_0(x), L_\ell(x), L_{\text{blind}}(x)$.
4. Combine those expressions in Horner order with y . Compute the expected quotient evaluation and the commitment to h_x .
5. Construct the complete opening-query list. Reproduce the point-set groups, interpolants and linear commitment combinations. Compute the expected final evaluation at x_3 .

6. Read and verify the masked inner-product opening. Supply the final challenge-dependent generator contribution and evaluate the entire multiscalar multiplication to $\mathcal{O}$.

The surrounding application supplies proof framing and enforces its specified length or end-of-input rule. The generic transcript reader consumes the elements it needs; it does not by itself establish that an arbitrary enclosing byte stream has no trailing data.

Evaluations are claims until the final opening check succeeds. A vanishing test performed on unauthenticated scalar messages is not already a verified circuit. Conversely, an opening proof authenticates a polynomial claim, not the application's intended semantics: the circuit and public-input binding remain essential.

8 Knowledge soundness and zero knowledge

8.1 What the algebra has established

The preceding derivations establish deterministic identities: interpolation represents columns, vanishing means divisibility, the permutation and lookup constraints enforce their advertised relations outside specified bad events, and every honest inner-product fold preserves the group equation. They therefore explain honest correctness and the structure of the verification test.

A malicious prover need not generate group elements from known coefficient vectors using the honest algorithm. For example, merely observing a point P does not reveal a polynomial and blind opening it. The expression $\text{Commit}(p; r) = P$ has many mathematical solutions. An *extractor* is an algorithm with specified access to the prover which obtains a suitable witness efficiently; saying that a solution exists is not a construction of such an algorithm.

This distinction is why the folding invariant is not, by itself, a knowledge-soundness proof. It checks what happens if the vectors used in the calculation exist with the requisite consistency. Extraction has to establish that consistency against an adversarial transcript.

8.2 A conditional arithmetic-layer statement

It is useful to isolate exactly what a secure commitment-and-opening layer must provide. Consider first an interactive version with independent uniform verifier challenges.

Assumption 8.1 (Consistent polynomial extraction). Except with probability $\varepsilon_{\text{open}}$, an extractor associates every committed polynomial with coefficients of the claimed degree, consistently across all its uses, and accepted openings have the corresponding evaluations. The extraction guarantee respects the challenge chronology: it supplies the relevant fixed polynomial objects before the later challenges to which the algebraic tests are applied.

This is a security requirement on the composed opening protocol, not a property proved by the Pedersen commitment's hiding or binding lemma. It also excludes interpreting a polynomial fitted after learning the test point as a polynomial fixed before that point.

Proposition 8.2 (Recovering a satisfying arithmetic witness). *Under consistent polynomial extraction, and outside polynomial-test, compression and zero-factor bad events, an accepted interactive proof yields advice values satisfying the circuit gates, copy equalities and lookup memberships.*

Proof. Evaluate the extracted advice polynomials on the usable rows of H . Authentication of all queried evaluations makes the verifier's computed numerator the value of the actual combined constraint polynomial. Outside the random-point identity-test event, $N_y = (X^n - 1)h$ as polynomials. Outside the random-combination event, each constituent constraint therefore vanishes on H .

The permutation recurrences telescope. Their initial value is one, their chunk links connect the products, and nonzero factors make the Boolean final value equal one. Outside the labelled-product collision event, the permutation lemma implies all copy equalities.

The lookup products likewise prove that A' and S' have the same multisets as the compressed input and table. The first-row match and run constraints then place every compressed input in the compressed table. Outside tuple-compression collisions, that is membership of the original tuples. The custom gates already vanish at every required row. The extracted advice values are the required circuit witness. $\square$

All algebraic error terms can be bounded using the elementary root bound. For example, for M combined constraints the y test contributes at most $(M-1)/|\mathbb{F}|$. A committed degree-less-than- $(d-1)n$ quotient makes the final difference have degree at most $dn - 1$, giving at most $(dn - 1)/|\mathbb{F}|$ for the x test. With C participating copy cells, a nonzero labelled-product difference has total degree at most C in β, γ . Each zero factor has probability at most $1/|\mathbb{F}|$; count all numerator and denominator factors. Lookup compression must count all distinct input-table tuple pairs, not merely one pair. A union bound combines these contributions and $\varepsilon_{\text{open}}$. Excluded challenges such as $x \in H$ must be included in the failure accounting as well.

These are deliberately modular bounds, not an asserted concrete security level for an entire production proof. The application's circuit-to-relation implication is a further step after extracting a satisfying circuit witness.

8.3 The commitment and Fiat-Shamir boundary

The usual analysis of this family of inner-product commitments uses the discrete-logarithm assumption and a model controlling adversarial group computations. In the *algebraic group model*, an adversary that outputs a group element must also supply, to the analysis, a linear representation of it in previously available group elements. The verifier does not receive these representations. They are an extra restriction on the analysed adversary, not a consequence of canonical point decoding or of the group being cyclic.

Such representations let an extractor compare generator coefficients in suitable related accepting transcripts. Unknown efficient relations between independently generated points prevent inconsistent representations except with the model's failure probability. Constructing the extractor, its rewinding strategy and its time bound is a separate theorem obligation. The *Crypto Guide*, "Knowledge, simulation, and Schnorr", demonstrates these distinctions for a simpler three-message protocol; that demonstration is not automatically a theorem for this longer protocol.

Fiat-Shamir replaces interactive challenges by transcript hashes. Its analysis models the hash as an oracle with consistent random answers on new inputs and counts the adversary's queries. The adversary can try different commitment prefixes and observe their challenges. Consequently, an interactive one-attempt root bound cannot simply be relabelled as the noninteractive security bound. Rewinding or oracle programming, when used by a proof, must be justified within its exact model, including adaptively selected public inputs and correlated openings.

The Halo paper supplies the originating inner-product construction and its modelled security analysis. The pinned implementation includes a masked variant and the concrete reductions described here. We do not assert that a short folding identity or an informal appeal to that paper proves every detail of this implementation's complete composition. The explicit extraction assumption above states the remaining boundary instead of presenting it as a proved consequence of binding. This boundary matters when using the argument as a security premise for the application.

8.4 Why polynomial evaluations need masks

A hiding commitment does not hide a subsequently disclosed evaluation. If a witness polynomial were determined entirely by its active rows, each evaluation could reveal a linear equation in those rows. Halo 2 therefore randomises unused row values as well as commitment blinds.

Fix a set B of s independently randomised domain points and fix all other row values. Let

$$T_B(X) = \prod_{h \in H \setminus B} (X - h).$$

The possible masked column polynomials form the affine space

$$a_0(X) + T_B(X)r(X), \quad \deg r < s.$$

Choosing the s row masks uniformly chooses r uniformly: both parameterisations describe the same s -dimensional affine space bijectively.

For $t \leq s$ fixed distinct query points $z_1, \dots, z_t$ outside H , the evaluation vector of the mask is

$$(T_B(z_1)r(z_1), \dots, T_B(z_t)r(z_t)).$$

Interpolation makes $r \mapsto (r(z_1), \dots, r(z_t))$ surjective, and all the factors $T_B(z_i)$ are nonzero. The *Math Guide*, “Linear algebra and uniform masks”, therefore implies that this vector is uniform on $\mathbb{F}^t$. It hides the corresponding evaluations of a_0 jointly, for these fixed points and this isolated linear observation.

The implementation chooses

$$b = \max(3, q_{\text{adv}}) + 2,$$

where q_{adv} is the maximum number of distinct configured rotations of any advice column, taking one as the fallback when there are no advice columns. The minimum three covers permutation polynomials queried at three rotations. One additional factor covers the multi-opening point x_3 , and one is an extra margin. Advice and permuted lookup columns actually randomise $b + 1$ rows; product polynomials randomise b . This accounts for the concrete row budget rather than merely saying that unspecified blinding is added.

8.5 The quotient mask and the joint transcript

The quotient depends nonlinearly on witness columns and auxiliary products. Its evaluation $h(x)$ is already determined by the constraint evaluations, but an additional value at x_3 could disclose new information. The independent random polynomial R is included with h_x in the group of polynomials queried at x . Their later Horner-combined evaluation at x_3 contains a random contribution from $R(x_3)$.

For fixed distinct x, x_3 and $n \geq 2$, the pair $(R(x), R(x_3))$ is uniform in $\mathbb{F}^2$. After revealing the first coordinate, the second remains uniform. When its batching coefficient is nonzero, it can therefore mask another scalar in that linear combination. The PCS mask s separately masks the final opened polynomial while preserving its required evaluation.

Neither observation proves full zero knowledge in isolation. Different messages can share masks, challenges are derived from earlier commitments, and later openings must remain consistent with those commitments. Two individually uniform messages can jointly reveal a witness, as the example $(r, r + w)$ shows. A complete zero-knowledge proof must construct one joint simulator for the entire view, including commitments, all evaluations, quotient reductions, inner-product messages and hash-derived challenges. If it relies on programming the hash oracle, that ability belongs in the statement of the theorem.

The local affine-mask and folding lemmas explain the mechanisms and their required dimensions. This volume does not replace the joint simulation obligation by those lemmas, nor does it claim that any missing composition theorem has thereby been proved.

8.6 The application-facing contract

To consume Halo 2 correctly, an application must fix the circuit and verification key, define the public-input mapping, frame and parse the proof, run the complete opening verification, and check every

non-circuit rule separately. It must establish that a satisfying circuit witness implies its intended relation, with canonical encodings and integer bounds where needed.

The resulting security claim is conditional on that circuit argument, the commitment and transcript security assumptions, correct parameter and key derivation, and correct implementation. Proof privacy does not hide the public input, the proof’s circuit shape, transaction metadata outside the circuit, or information the wallet discloses through its network activity. These are boundaries of the proved relation, not failures of the masking algebra.

8.7 Primary construction sources

The mechanism and byte order in this volume are checked against the pinned `halo2_proofs` source. Within that source, the relevant files are:

- `plonk/circuit.rs`, `plonk/prover.rs` and `plonk/verifier.rs`: circuit degree, blinding budget, complete message and query order.
- `plonk/permutation/` and `plonk/lookup/`: product recurrences, exact Boolean boundaries and rotations.
- `poly/domain.rs` and `plonk/vanishing/`: extended coset, random polynomial and quotient chunks.
- `poly/commitment.rs`, `poly/commitment/` and `poly/multiopen/`: generator derivation, asymmetric masked folding and Horner-ordered multi-opening reduction.
- `transcript.rs` and `plonk.rs`: challenge bytes and verification-key representative.

The Halo 2 design documentation explains the design; its sketches are not substituted for the concrete source where conventions differ. Bowe, Grigg and Hopwood’s *Halo: Recursive Proof Composition without a Trusted Setup* is the primary paper behind the commitment construction and modelled security analysis. Recursive composition itself is not needed to understand the single proof verified here.

The executable checks in `research/check_foundations.py` and `research/check_halo2.py` recompute the small-field example, folding identities and multi-opening algebra. They test the displayed calculations, not cryptographic hardness, witness extraction or a complete transcript simulation.